CHRONOCHEF: 4-MEMBER PITCHING SCRIPT & VIVA DEFENSE GUIDE

A Complete Guide for Pitching to Your Java Professor — Architecture, System Specs, Spoken Scripts, Q&A; & Codebase Reference

Project Title: ChronoChef — Smart Dynamic Pricing & Expiry Tracker	**Academic Defense Target:** 4-Member Oral Presentation & Viva
Course: PBCST304 — Object-Oriented Programming (S3)	**Tech Stack:** Java SE, Java Swing (MVC), JDBC, SQLite
Verification Status: 35/35 Test Suite Cases Passing (100% Verified)	**Recommended Total Time:** 11-12 Minutes Total (~2.5m / Member)

1. System Architecture & Hardware Requirements (Project Prerequisite)

Member 1 introduces these specifications during the opening presentation to demonstrate that ChronoChef was engineered with an ultra-lightweight, zero-bloat footprint, enabling community food banks and relief shelters to run reliably on budget or donated hardware without external database server overhead.

Component / Spec	Minimum Requirement	Recommended Specification	Engineering & Academic Rationale
Processor (CPU)	Intel Core i3 (4th Gen) / AMD Ryzen 3 / Dual-Core 2.0 GHz	Intel Core i5 / AMD Ryzen 5 or higher	Lightweight multithreading and event dispatching via the Swing Event Dispatch Thread (EDT).
Memory (RAM)	4 GB DDR3 / DDR4	8 GB DDR4	Minimal JVM heap allocation (~128MB to 256MB active memory under full operational load).
Storage (Disk)	250 MB free disk space	500 MB SSD	Zero-configuration embedded SQLite engine; entire relational schema is stored in one local file (chronochef.db).
Operating System	Windows 10/11, Ubuntu Linux 20.04+, macOS Catalina+	Any 64-bit OS with Java support	Cross-platform portability guaranteed by the Java Virtual Machine (WORA — *Write Once, Run Anywhere*).
Java Runtime	Java SE Runtime (JRE / JDK 11+)	OpenJDK / Oracle JDK 17 (LTS)	Standard library implementation using Java Swing, AWT, Event Delegation Model, and JDBC 4.2 API.
Display Resolution	1366 × 768 (HD)	1920 × 1080 (Full HD)	Responsive single-window CardLayout dynamically adjusts layout panels without visual truncation.

Key Engineering Advantages for Academic Defense:

- **Zero Server Dependencies:** Unlike web applications that require external MySQL/PostgreSQL servers and HTTP web servers, ChronoChef is fully self-contained.
- **100% Offline Resilience:** Operates without internet connectivity, making it ideal for disaster zones and food distribution camps.
- **Sub-Millisecond Query Response:** In-process JDBC queries execute instantaneously with zero network latency overhead.

2. Master 4-Member Allocation, Transitions & Pitching Strategy

Each member has a designated technical focus mapped directly to the OOP syllabus (Modules 1 to 4). The presentation avoids line-by-line syntax recitation and instead emphasizes **architecture, design patterns, mathematical modeling, and live execution**.

Member	Designated Role & Focus	Syllabus	Key Source Files	Time
Member 1	**System Specs, Problem Motivation & Core OOP** Hardware needs, Encapsulation, Hierarchical Inheritance, Dynamic Method Dispatch.	Mod 1 & Mod 2	model/FoodItem.java model/PerishableItem.java model/PackagedItem.java	2.5 min
Member 2	**Mathematics, Urgency Formulas & Sorting Algorithms** Days Remaining, Priority Distribution Index (PDI), Dynamic Subsidies, Bubble Sort.	Mod 1 & Mod 3	service/PriorityPricingService.java model/DistributionRecord.java	2.5 min
Member 3	**Persistence, Design Patterns & Food Safety Exceptions** Singleton pattern, DAO pattern, PreparedStatement security, Custom Checked Exceptions.	Mod 3 & Mod 4	database/DatabaseManager.java database/FoodBankDAO.java exception/*.java	2.5 min
Member 4	**Swing GUI Architecture, Live Demo & Social Impact** CardLayout, Event Delegation Model, 5 modules live checkout, automated test proof.	Mod 4	ui/MainFrame.java ui/InventoryPanel.java ui/DistributionPanel.java	3.0 min

Verbal Handoff Transitions (Memorize These Cue Lines)

Transition	Exact Spoken Handoff Cue to Recite
Member 1 → Member 2	*"Now that we have seen how our Object-Oriented domain entities represent the real world, my teammate [Member 2] will explain the mathematical algorithms and pricing rules that drive our inventory urgency."*
Member 2 → Member 3	*"Having explored our algorithm and pricing logic, my teammate [Member 3] will now discuss how this data is securely stored in SQLite and protected by custom exceptions."*
Member 3 → Member 4	*"Now, my teammate [Member 4] will take you through our Swing graphical user interface and demonstrate the live running application."*
Member 4 → Professor	*"We have verified all 35 project test assertions with a 100% pass rate. Our team is now ready for your questions. Thank you!"*

Presentation Rules of Engagement (How to Score Maximum Marks)

- **No Line-by-Line Code Reading:** Explain *what* the system accomplishes and *which OOP principle* makes it possible. Professors deduct marks for reading syntax.
- **Maintain Eye Contact:** Speak directly to your professor while making gestures toward the slide or application.
- **Strict Time Discipline:** Never exceed 3 minutes per member so the evaluator has ample time to conduct the viva defense.

3. Member 1: Problem Motivation, System Specs & Core OOP Modeling

Assigned Files: Main.java, model/FoodItem.java, model/PerishableItem.java, model/PackagedItem.java | Time: 0:00 – 2:30

*"Respected Professor and evaluators, good morning. Today, our team is presenting **ChronoChef** — an intelligent desktop food inventory and dynamic subsidy management system developed in Java.*

*To understand why we built ChronoChef, let us look at the operational reality of charitable food banks. Food banks handle massive volumes of donated groceries with varying expiration dates. Tragically, over **30% of donated perishable inventory spoils on warehouse shelves** before it can reach needy families. The reasons are static paper logs, lack of urgency-based subsidies, and the human risk of distributing expired stock.*

ChronoChef eliminates this waste. It autonomously computes remaining shelf-life, assigns a real-time urgency score, subsidizes near-expiry food up to 100% free distribution, and strictly blocks expired goods from being handed out.

*Before looking at the code structure, I want to emphasize the **System Requirements**. Because relief shelters operate on budget or donated machines, we engineered ChronoChef with an ultra-lightweight footprint:*
*• On hardware: It runs smoothly on an entry-level **Intel Core i3 or AMD Ryzen 3 CPU with just 4 GB of RAM** and under 250 MB disk space.*
*• On software: It requires **Java SE 11 or higher** and uses an embedded zero-configuration database, making it 100% offline-ready.*

*In our core data tier, we strictly implemented **Key Object-Oriented Principles**:*
*1. **Encapsulation:** Our base entity, FoodItem, strictly protects internal state with private attributes and public getters and setters.*
*2. **Hierarchical Inheritance:** Food categories have distinct real-world physical properties. Therefore, FoodItem is extended by two specialized subclasses: PerishableItem — which encapsulates cold-chain storage temperatures — and PackagedItem, which tracks manufacturer batch lot numbers.*
*3. **Dynamic Method Dispatch:** Each subclass overrides the polymorphic getStorageInstructions() method. At runtime, the JVM dynamically resolves whether to return chilled refrigeration instructions or dry pantry instructions based on the actual object instance.*

Now that we have seen how our Object-Oriented domain entities represent the real world, my teammate [Member 2] will explain the mathematical algorithms and pricing rules that drive our inventory urgency."

Professor Viva Questions & Model Answers for Member 1

Q1: Why did you use inheritance instead of simply putting an 'isPerishable' boolean flag inside FoodItem?

Answer: Putting a flag inside FoodItem violates the Single Responsibility Principle and causes code pollution with messy if-else checks whenever storage behavior or attributes differ. By using Hierarchical Inheritance, PerishableItem and PackagedItem maintain their own distinct attributes (such as optimumTempCelsius vs batchNumber) and cleanly override behavior through polymorphism.

Q2: How does the Java Virtual Machine (JVM) resolve Dynamic Method Dispatch in your code?

Answer: When getStorageInstructions() is called on a FoodItem reference, Java verifies at compile time that the method exists in the superclass interface. At runtime, however, the JVM inspects the object's virtual method table (vtable) and dispatches the call directly to the overriding method of the actual runtime subclass instance.

Q3: Why build a standalone Java desktop application instead of a web app for this use case?

Answer: Food banks, disaster relief camps, and emergency shelters often operate in areas with unstable or non-existent internet connectivity. A Java Swing desktop app with an embedded SQLite database guarantees 100% offline functionality, instantaneous sub-millisecond response times, and zero server hosting expenses.

4. Member 2: Mathematical Logic, Urgency Formulas & Sorting Algorithms

Assigned Files: service/PriorityPricingService.java, model/DistributionRecord.java | Time: 2:30 – 5:00

"Thank you, [Member 1]. Good morning, Professor. I will explain the mathematical logic, automated decision formulas, and sorting algorithms that power ChronoChef.

*In a food bank, inventory cannot be treated as a simple First-In, First-Out queue. Fresh milk received today that expires in 2 days must be prioritized far ahead of canned beans received a month ago that expire next year. To solve this, we formulated the **Priority Distribution Index (PDI)**.*

Our engine evaluates two primary mathematical formulas:
*1. **Days Remaining Calculation (D):** We compute the delta between the expiry timestamp and the current system timestamp converted to days:*
*Days Remaining (D) = (ExpiryDate_ms - CurrentDate_ms) / (1000 * 60 * 60 * 24)*
If D is negative, the item is mathematically flagged as expired.

*2. **Priority Distribution Index (PDI Score):** For non-expired items, we map shelf life into a normalized 1-to-100 urgency score using:*
*PDI = max(1, 100 - (D * 2))*
*• If an item has **5 days or fewer**, its PDI reaches **90 to 100**, classifying it as **URGENT**.*
*• 6 to 15 days maps to **HIGH** priority.*
*• 16 to 30 days maps to **MEDIUM** priority.*
*• Anything over 30 days is categorized as **LOW** priority.*

*3. **Dynamic Rule-Based Subsidies:** To incentivize community shelters to clear stock before it spoils, our pricing engine applies automatic subsidies:*
*• Items within **2 days of expiry** receive a **100% Subsidy (distributed completely FREE - ■0.00)**.*
*• Items within **3 to 7 days** receive a **50% Subsidy**.*
• Items beyond 7 days remain at their standard base price.

*4. **Algorithm & Sorting:** To display inventory strictly by urgency, our service implements an in-memory **Bubble Sort algorithm** directly on a Vector<FoodItem>. It iterates through adjacent items and sorts them in descending order of their PDI scores. Because a local shelter manages around 500 to 1,000 active inventory items, this classic O(n²) sorting technique completes in just a few milliseconds on 4 GB RAM while perfectly demonstrating textbook data structure manipulation.*

Having explored our algorithm and pricing logic, my teammate [Member 3] will now discuss how this data is securely stored in SQLite and protected by custom exceptions."

Professor Viva Questions & Model Answers for Member 2

Q1: Why did you use Bubble Sort instead of built-in Collections.sort() or Arrays.sort()?

Answer: While Collections.sort() implements TimSort with O(n log n) efficiency, we intentionally implemented the classic Bubble Sort algorithm to demonstrate fundamental computer science sorting mechanics directly on Java Collections (Vector), aligning with university course guidelines. For standard food bank batch sizes of under 1,000 items, execution time is negligible.

Q2: What happens mathematically when an item's expiration date has already passed (D < 0)?

Answer: When D < 0, calculatePdiScore() short-circuits and immediately returns 0, tagging the item with the EXPIRED enum. This acts as an automated quarantine flag that prevents the item from entering the subsidy calculation or distribution pipelines.

Q3: Could an operator tamper with the system clock to manipulate PDI scores?

Answer: While the formula evaluates against system time, date integrity is strictly validated upon donation intake via ISO YYYY-MM-DD regular expression patterns and calendar bounds, preventing corrupted timestamps from entering the system.

5. Member 3: Persistence Layer, Design Patterns & Food Safety Exceptions

Assigned Files: database/DatabaseManager.java, database/FoodBankDAO.java, model/InventoryOperations.java, exception/*.java | Time: 5:00 – 7:30

"Thank you, [Member 2]. Good morning, Professor. I will present our persistence tier, software design patterns, and our robust food safety exception handling mechanism.

*For data persistence, ChronoChef utilizes **JDBC with SQLite**. We chose an embedded SQLite database because it creates a portable, self-contained database file (chronochef.db) stored directly in the project directory, eliminating the need for external database servers or complex network configurations.*

To manage database operations cleanly, we incorporated two proven software design patterns:
*1. **Singleton Pattern:** Opening database connections is computationally expensive and uncontrolled connections can lock SQLite database files. In DatabaseManager.java, we implemented a private constructor and a static synchronized getInstance() method. This guarantees that exactly one connection pipeline exists throughout the entire JVM lifecycle.*
*2. **Data Access Object (DAO) Pattern:** We decoupled the database from the user interface by defining an interface: InventoryOperations. Our DAO class, FoodBankDAO, implements this contract to handle all CRUD queries, category filtering, and transaction logging.*
*3. **SQL Injection Prevention:** We strictly avoided raw string concatenation in SQL queries. Every single database query uses parameterized PreparedStatement objects. The SQL structure is pre-compiled by the database engine and user inputs are bound strictly as data literals, neutralizing SQL injection attacks.*

*Next, I want to emphasize **Food Safety & Custom Exceptions**:*
• In a commercial grocery store, an expired sale is a financial error; in a food bank distributing to vulnerable shelters, giving out expired food is a hazardous safety violation.
• To enforce safety at the compiler level, we created two custom checked exceptions:
* • ExpiredItemDistributionException: Thrown if an operator attempts to distribute an item with D < 0.*
* • InvalidQuantityException: Thrown if an operator attempts to dispatch more stock than physically exists in warehouse inventory.*
*• Because these are **checked exceptions** inheriting directly from java.lang.Exception, the Java compiler enforces strict try-catch handling. The distribution transaction cannot execute without explicit exception handling, safety rollback, and operator alerts.*

Now, my teammate [Member 4] will take you through our Swing graphical user interface and demonstrate the live running application."

Professor Viva Questions & Model Answers for Member 3

Q1: Why did you implement DatabaseManager as a Singleton instead of opening a 'new Connection()' in every DAO method?

Answer: Creating connections on every query creates massive CPU and memory overhead due to repeated file descriptor handshakes. Furthermore, SQLite is an embedded file database; concurrent unmanaged connections can lead to SQLITE_BUSY file locking errors. The Singleton pattern guarantees centralized connection lifecycle management and controlled access.

Q2: Why make ExpiredItemDistributionException a checked exception instead of an unchecked RuntimeException?

Answer: Checked exceptions enforce **compile-time safety**. An unchecked RuntimeException can be overlooked by a developer and crash the app during production. By making it checked, the compiler forces the UI controller to wrap distribution calls in a try-catch block, ensuring safety alerts and rollbacks are always executed.

Q3: How do you prevent JDBC connection leaks and cursor exhaustion?

Answer: Every database operation uses Java's **try-with-resources** or explicit finally blocks to close all ResultSet and PreparedStatement objects immediately after execution, releasing OS file handles even if an unexpected SQL error occurs.

6. Member 4: Swing GUI Architecture, Live Demo & Social Impact

Assigned Files: ui/MainFrame.java, ui/InputPanel.java, ui/InventoryPanel.java, ui/ManagementPanel.java, ui/DistributionPanel.java | Time: 7:30 – 10:30

"Thank you, [Member 3]. Good morning, Professor. I will present our Swing UI architecture and walk you through the live application.

*When designing the graphical user interface, our objective was to build a clean, modern, and reliable dashboard using **pure Java Swing and AWT**, strictly complying with the college syllabus.*

Our UI architecture is grounded in three core design decisions:
*1. **Single-Window CardLayout Pattern:** Instead of opening multiple messy pop-up windows (JFrames) that clutter the desktop, we host all 5 system views inside a single MainFrame using CardLayout. A top navigation bar switches between panels with zero screen flicker.*
*2. **Delegation Event Model:** We cleanly decouple UI components from backend logic. Buttons register ActionListener instances that capture ActionEvents and delegate execution to service methods.*
*3. **Layout Hierarchy:** We combined nested layout managers — BorderLayout for main frames, GridLayout for structured forms, and FlowLayout for toolbars — ensuring consistent rendering across screen resolutions.*

(Member 4 now transitions to the live demo screen)

*Allow me to demonstrate the **5 Core Modules** in action:*
*• **Module 1 — Data Input Panel:** Here we register incoming food donations. Form validation prevents blank fields, negative numbers, and malformed dates.*
*• **Module 2 — Live Inventory & Expiry Tracker:** This table renders live stock with color-coded urgency badges — **Red for Urgent**, **Orange for High**, and **Green for Medium**. Clicking 'Sort by PDI' triggers our Bubble Sort algorithm, instantly bubbling urgent perishable goods to the top of the operator's view.*
*• **Module 3 — Management Panel:** Allows staff to select existing stock, restock new shipments, or write off damaged goods.*
*• **Module 4 — Distribution & Subsidy Billing:** This is our checkout station. When we select near-expiry Milk (2 days remaining) and dispatch it to a shelter, watch the system automatically apply a **100% Zero-Waste Subsidy (Total: ■0.00)**, deduct stock safely in SQLite, and generate an itemized text receipt.*
Now watch what happens if we attempt to distribute an expired item: the system intercepts the action, throws our ExpiredItemDistributionException, and displays a red safety modal, strictly blocking the checkout.
*• **Module 5 — Analytics & Social Impact:** Displays 6 aggregated metrics: Total Inventory Count, Urgent Items Count, Total Food Waste Prevented in Kilograms, and Total Community Subsidies Granted.*

*In summary, ChronoChef combines **core Object-Oriented principles, deterministic mathematical algorithms, secure database design, and an intuitive Swing interface** to solve a critical real-world problem: ending food waste and hunger in our communities.*

Our automated test suite has verified all 35 project test assertions with a 100% pass rate. We are now ready for your questions. Thank you!"

Professor Viva Questions & Model Answers for Member 4

Q1: Why did you use CardLayout instead of opening multiple JFrames or using JTabbedPane?

Answer: Opening multiple JFrame windows causes desktop clutter and consumes extra OS window handles. While JTabbedPane works, CardLayout paired with a custom navigation toolbar gives us full programmatic control over view transitions, lazy loading, and automatic table re-queries whenever a panel is brought to the front.

Q2: What thread does Swing use to paint components and handle button clicks?

Answer: Swing components are not thread-safe and must be manipulated strictly on the **Event Dispatch Thread (EDT)**. In our Main.java, we launch the interface using SwingUtilities.invokeLater(() -> { ... }) to ensure the GUI is initialized on the EDT, preventing deadlocks and race conditions.

Q3: How does InventoryPanel update its table when a new item is added in InputPanel?

Answer: When an item is added, the DAO saves the record in SQLite. When the user navigates to the Inventory view, our transition listener triggers refreshTableData(), which re-queries the DAO, clears the DefaultTableModel vector, repopulates rows, and calls fireTableDataChanged() to re-render the table with fresh PDI urgency badges.

7. Examiner Cross-Defense & Trap Questions

Professors frequently ask cross-cutting questions to test whether every team member understands how the entire system connects together. Below are the top cross-defense questions with complete model answers.

TRAP Q1: Trace the lifecycle of a single button click from GUI to Database and back.

Answer: 1. **User Action (View - Member 4):** Operator clicks 'Confirm Distribution' in DistributionPanel.
2. **Event Handling (Member 4):** actionPerformed() fires and validates user input fields.
3. **Domain Check (Model - Member 1):** The selected FoodItem is checked for days remaining (D).
4. **Safety Exception (Member 3):** If D < 0, ExpiredItemDistributionException is thrown and caught, stopping the transaction.
5. **Pricing Math (Member 2):** If D ≥ 0, PriorityPricingService calculates dynamic subsidy.
6. **Persistence (Member 3):** FoodBankDAO uses the Singleton DatabaseManager and a PreparedStatement to update stock in food_items and log a row in distributions.
7. **GUI Refresh (Member 4):** mainFrame.refreshAllViews() re-queries SQLite and triggers fireTableDataChanged() across all tables.

TRAP Q2: Why did you choose SQLite over MySQL or PostgreSQL for this project?

Answer: SQLite is an embedded, serverless relational database engine stored as a single disk file (chronochef.db). Community food banks and relief shelters often operate in bandwidth-constrained or offline environments without dedicated IT staff or database servers. SQLite requires zero configuration, embeds cleanly via a standalone JAR, and guarantees full ACID compliance with zero deployment overhead.

TRAP Q3: What happens if two volunteers run ChronoChef simultaneously on the same machine?

Answer: Because our DatabaseManager is implemented as a synchronized Singleton, all threads within the JVM share one synchronized connection pipeline. If two separate JVM processes attempt to write to the SQLite file concurrently, SQLite's built-in file-locking mechanism serializes write transactions, preventing database corruption.

TRAP Q4: How do you prove that your system is thoroughly tested and bug-free?

Answer: ChronoChef includes an automated unit test suite (ProjectTestRunner.java) executing **35 comprehensive tests** across all modules. It validates OOP inheritance, polymorphism, Bubble Sort ordering, PDI boundary conditions (D < 0, D = 0, D > 30), custom exception throwing, and SQLite DAO CRUD operations, achieving a verified 100% pass rate.

Presentation Day Emergency Checklist

- ✓ **Compile & Run:** Double-click run.bat or compile.bat prior to presentation to ensure zero build errors.
- ✓ **Automated Test Proof:** Run run_tests.bat and show the console output: [35 / 35 PASS - 100%] if asked about testing.
- ✓ **Sample Data Ready:** Ensure chronochef.db contains at least 1 Urgent item, 1 Expired item, 1 Perishable item, and 1 Packaged item for the demo.
- ✓ **Time Discipline:** Keep each member's speech strictly between 2.5 and 3 minutes so the total presentation wraps up under 11 minutes.

8. Project Directory Structure & Folder Anatomy

A rigorous explanation of every folder in the project workspace, distinguishing between build outputs, runtime libraries, persistent database stores, and source package structures.

Folder / Directory Path	Category / Type	Architectural Role & Contents
project code/	Root Container	Top-level directory housing the complete executable project, batch scripts, and local database file.
project code/src/	Source Root	Root directory for all uncompiled Java source files (.java), organized into modular, domain-driven package namespaces.
project code/bin/	Build Artifacts	Destination folder where the Java compiler (javac) outputs compiled bytecode (.class files), preserving package structures.
project code/lib/	External Dependencies	Contains external runtime JAR archives: sqlite-jdbc-3.45.2.0.jar (embedded SQLite driver) and SLF4J logging bridge.
src/database/	Persistence Package	Handles low-level JDBC database connections, connection pooling via Singleton, data access objects (DAO), and automated mock data seeders.
src/model/	Domain Entity Package	Houses core OOP data models (FoodItem, PerishableItem, PackagedItem, DistributionRecord) and interface contracts.
src/service/	Business Logic Package	Contains algorithmic evaluation engines: Days Remaining calculations, Priority Distribution Index (PDI), dynamic subsidy tiers, and Bubble Sort.
src/exception/	Custom Exception Package	Defines custom checked exceptions (ExpiredItemDistributionException, InvalidQuantityException) enforcing compile-time safety.
src/ui/	Presentation GUI Package	Hosts all Java Swing views, custom cards, navigation toolbars, table renderers, and event listeners built with pure Swing and AWT.
src/test/	Verification Package	Automated test harness executing 35 unit assertions and diagnostic UI capture tools verifying 100% codebase correctness.

Root Batch Launcher Scripts:

- **compile.bat:** Invokes javac with the classpath set to lib/*, compiling all 20 source files from src/ into bin/.
- **run.bat:** Executes the compiled application entry point Main using java -cp "bin;lib/*" Main.
- **run_tests.bat:** Runs the automated test suite ProjectTestRunner, executing 35 unit assertions with console pass/fail summaries.

9. Complete Class Reference Guide (Part 1: Core, Model, DB & Service)

A definitive explanation of every class in each project package, detailing its specific technical role, OOP mechanism, and practical use in ChronoChef.

Package / Class	Type & Inheritance	Technical Role, Purpose & Practical Use
Main.java ***(Root Package)***	Entry Point Class	**Bootstrap Coordinator:** Initial entry point of ChronoChef. Invokes database initialization, verifies table schemas, runs DataSeeder if records are empty, and initializes MainFrame strictly on the Swing Event Dispatch Thread (EDT) using SwingUtilities.invokeLater().
model/ FoodItem.java	Base Superclass (Encapsulated)	**Core Domain Entity:** Encapsulates common attributes of donated food items (ID, name, category, quantity, basePrice, expiryDate, receivedDate). Features overloaded constructors, parameter validation, and defines the polymorphic base method getStorageInstructions().
model/ PerishableItem.java	Subclass extends FoodItem	**Cold-Chain Goods:** Specializes FoodItem for dairy, meat, and fresh produce. Adds the private attribute optimumTempCelsius and overrides getStorageInstructions() to supply mandatory refrigeration guidelines (e.g. 'Keep refrigerated between 2°C and 4°C').
model/ PackagedItem.java	Subclass extends FoodItem	**Dry / Canned Goods:** Specializes FoodItem for dry pantry goods (canned vegetables, rice, cereals). Adds batchNumber for manufacturer recall audits and overrides getStorageInstructions() for dry pantry room-temperature storage.
model/ DistributionRecord.java	Transaction Audit Entity	**Distribution Transaction Model:** Immutable representation of an itemized food clearance event. Tracks transaction ID, food item reference, quantity issued, recipient shelter name, base price, dynamic subsidy percentage, effective price, and dispatch timestamp.
model/ InventoryOperations.java	Interface Contract	**Abstract Operational Contract:** Declares standard abstract methods for CRUD actions, category searches, restocking, distribution transactions, and analytics aggregation (addItem, getAllItems, recordDistribution).
database/ DatabaseManager.java	Singleton Pattern (Thread-Safe)	**Central Connection Manager:** Ensures only one active connection manager pipeline exists across the JVM lifecycle. Features a private constructor, a static synchronized getInstance() method, and safe connection lifecycle and cleanup methods.
database/ FoodBankDAO.java	DAO Pattern implements InventoryOperations	**Data Access Object:** Bridges Java objects and SQLite. Pre-compiles SQL using parameterized PreparedStatement to prevent SQL injection, maps relational rows (ResultSet) to Java entities, and handles transactional stock deductions.
database/ DataSeeder.java	Mock Data Utility	**Automated Seed Generator:** Creates SQLite tables (food_items, distributions) if absent, and populates 10 realistic sample food items across Urgent, High, Medium, and Expired categories for instant evaluation.
service/ PriorityPricingService.java	Business Logic Service	**Computational Engine:** Evaluates Days Remaining (D), computes Priority Distribution Index (PDI), applies rule-based dynamic subsidies (100% free / 50% discount), and implements in-memory Bubble Sort directly on Vector<FoodItem>.

9. Complete Class Reference Guide (Part 2: Exceptions, UI & Test)

Continuing the class-by-class technical breakdown covering custom safety exceptions, Swing UI MVC panels, and automated quality assurance test harnesses.

Package / Class	Type & Base Class	Technical Role, Purpose & Practical Use
exception/ ExpiredItemDistributionException.java	Checked Exception extends Exception	**Food Safety Invariant Enforcer:** Thrown when an operator attempts to distribute an item with $D < 0$. Because it is a checked exception, the Java compiler mandates explicit try-catch handling, preventing expired food distribution.
exception/ InvalidQuantityException.java	Checked Exception extends Exception	**Inventory Bounds Enforcer:** Thrown when user enters non-positive quantities (≤ 0) or attempts to distribute more items than currently exist in stock, preventing negative inventory balances.
ui/ MainFrame.java	Swing Frame Container extends JFrame	**Single-Window Shell:** Master window using CardLayout to manage the 5 primary views. Features a unified top navigation toolbar, window listener hooks, and coordinates global table refreshes across views without opening multiple windows.
ui/ InputPanel.java	Swing Panel View extends JPanel	**Module A (Intake):** Form view for registering donated items. Enforces field validation (empty check, number formats, ISO YYYY-MM-DD date pattern) and dispatches clean objects to FoodBankDAO.
ui/ InventoryPanel.java	Swing Panel View extends JPanel	**Module B (Expiry Tracker):** Live table dashboard with real-time text search, priority category filters, custom table cell renderers showing urgency color badges (Red/Orange/Green), and a button triggering Bubble Sort.
ui/ ManagementPanel.java	Swing Panel View extends JPanel	**Module C (Maintenance):** Inventory maintenance terminal. Allows staff to select existing stock, restock incoming donations, update price or storage attributes, or write off damaged stock.
ui/ DistributionPanel.java	Swing Panel View extends JPanel	**Module D (Subsidy Checkout):** Point-of-distribution terminal. Calculates dynamic subsidies, catches safety exceptions, updates SQLite inventory, and prints itemized ASCII distribution receipts.
ui/ AnalyticsPanel.java	Swing Panel View extends JPanel	**Module E (Impact Analytics):** Executive reporting view. Renders 6 metric cards (total items, urgent items, food waste prevented in kg, total subsidies granted) and an audit table of historical distributions.
test/ ProjectTestRunner.java	Automated Test Suite	**Verification Harness:** Executes 35 standalone unit assertions across all packages. Validates inheritance, polymorphism, PDI boundary mathematics, Bubble Sort ordering, safety exceptions, and SQLite DAO queries (100% pass rate).
test/ CaptureScreenshots.java	Diagnostic Utility	**Automated UI Capture:** Programmatically initializes GUI frames off-screen and saves high-resolution PNG screenshots of all 5 operational panels for documentation and viva presentations.

Codebase Architecture Summary for Professor Evaluation:

- **High Cohesion, Low Coupling:** The UI classes know only about interfaces and data transfer objects; they never perform direct raw SQL operations.
- **Extensibility:** New food types (e.g. FrozenItem, PreparedMealItem) can be introduced simply by extending FoodItem and overriding getStorageInstructions() without modifying existing code (Open/Closed Principle).
- **Auditability:** Every item issue is tracked in an immutable distribution audit log, ensuring complete transparency for charitable donors.